Dashboard Page — Information Document

Page Overview

Purpose

The Dashboard is the landing page of resmon (route `/` , rendered by `DashboardPage.tsx`). It gives an at-a-glance view of the user's automated surveillance state and recent activity. It has four visual regions rendered top-to-bottom:

1. A **"Welcome to resmon"** hero and an **"About resmon"** feature grid summarizing the six headline capabilities (Deep Dive, Deep Sweep, Routines, AI Summarization, Email & Cloud, Local-first).
2. A collapsible **`PageHelp`** panel (storage key `dashboard`) documenting what the page shows and what the user can do.
3. A **`CloudSyncCard`** showing resmon-cloud sign-in state.
4. An **Active Routines** table and a **Recent Activity** table.

The Dashboard is read-oriented; it does not create, edit, or delete routines, nor does it launch new executions. Its only outbound mutating call is the per-row export action on the Recent Activity table.

Primary User Flows

Three flows are supported directly from this page:

1. **Review surveillance state.** On mount, the page fetches the current routine list and the 10 most recent executions, filters the routines to those with `is_active` truthy, and renders them in the Active Routines table.
2. **Jump to a full report.** Clicking `View Report` on a Recent Activity row navigates to `/results?exec=<id>` , which opens that execution on the Results & Logs page.
3. **Export an execution bundle.** Clicking `Export` on a Recent Activity row posts `{ ids: [id] }` to `/api/executions/export` , receives a `.zip` path, shows a success banner with that path for 10 seconds, and (on Electron) exposes a `Reveal in Finder` / `Reveal in File Explorer` button that calls `window.resmonAPI.revealPath(exportPath)` .

A fourth flow — **Cancel a running execution** — is surfaced conditionally: if the row's execution is the current `activeExecution` from `ExecutionContext` and its status is `running` or `cancelling`, the action cell swaps `View Report` / `Export` for `View Monitor` (anchor to `#/monitor`) and a `Cancel` button wired to `cancelExecution(e.id)` from `ExecutionContext`. While `status === 'cancelling'`, the Cancel button disables and shows `"Stopping…"` with a spinner.

Inputs and Outputs

Inputs (observed in `DashboardPage.tsx`):

- `GET /api/routines` — full routine list; the page filters to `is_active` truthy client-side.
- `GET /api/executions?limit=10` — the 10 most recent executions.
- `useExecution()` context supplies `activeExecution`, `cancelExecution`, and `completionCounter`.
- `window.resmonAPI.platform` and `window.resmonAPI.revealPath` — optional Electron bridge used for the reveal button.

Outputs:

- `POST /api/executions/export` with body `{ ids: [<id>] }` — triggered by the per-row Export button.
- `POST /api/executions/{exec_id}/cancel` — indirectly, via `cancelExecution` from `ExecutionContext` when the row is the active running execution.
- Client-side navigation to `/results?exec=<id>` via `useNavigate`.

Known Constraints or Permissions

- The Recent Activity list is fixed to the 10 most recent executions (`?limit=10`); there is no pagination or filter UI on this page. Full filtering lives on the Results & Logs page.
- The Active Routines table only shows routines with `is_active` truthy; inactive routines are hidden here.
- The `Reveal` button only renders when `window.resmonAPI?.revealPath` is defined, i.e., when the page is running inside the Electron shell. In a plain-browser context, the path is shown but the reveal affordance is hidden.
- The reveal label is `Reveal in Finder` when `window.resmonAPI?.platform === 'darwin'`, otherwise `Reveal in File Explorer`.
- The export success and error banners auto-dismiss after 10,000 ms (`setTimeout` in `handleExportOne`).
- The page does not itself require cloud sign-in. `CloudSyncCard` handles cloud-account presentation independently.

- **Recent Activity is local-only.** The Dashboard calls `GET /api/executions?limit=10`, which returns rows from the local `executions` table via `get_executions`; it does **not** call `/api/executions/merged`. Cloud-only executions therefore do not appear in the Dashboard's Recent Activity table (they are reachable from the Results & Logs page, which is the caller of the merged endpoint). Each Recent Activity row does still carry an optional `execution_location` field, but on this endpoint it is not set by the backend; the frontend defaults `loc = e.execution_location ?? 'local'` and renders a `Local` badge accordingly.

Frontend

Route and Main Component

- **Route:** `/` (Dashboard is the default landing page registered in `App.tsx`'s router shell).
- **Main component:** `DashboardPage` (default export of `frontend/src/pages/DashboardPage.tsx`), a functional component using React hooks.

Child Components and Hooks

Child components rendered from `DashboardPage.tsx`:

- `PageHelp` — the standard collapsible page-help block (`storageKey="dashboard"`) with two sections, "What you see here" and "What you can do".
- `CloudSyncCard` (`components/Cloud/CloudSyncCard`) — displays resmon-cloud connection status.
- Two inline `<table class="simple-table">` blocks — Active Routines and Recent Activity.

Hooks and context:

- `useState` for `routines`, `executions`, `exportPath`, `exportError`.
- `useEffect` keyed on `completionCounter` — refetches `/api/routines` and `/api/executions?limit=10` on mount and every time an execution completes (see *Key Interactions and Events*).
- `useExecution()` from `context/ExecutionContext` — returns `{ activeExecution, cancelExecution, completionCounter }`. Per `ExecutionContext.tsx`, `completionCounter` is bumped whenever an execution reaches a terminal state (`completed`, `failed`, or `cancelled`), and that bump drives the Dashboard's refresh effect.
- `useNavigate()` from `react-router-dom` — used by `handleViewReport` to push `/results?exec=<id>`.
- `apiClient` (`api/client`) — wraps the backend REST surface.

Badge-palette helpers defined at module scope:

- `typeBadgeClass(t)` — maps `deep_dive` / `dive` → `badge-type-dive`, `deep_sweep` / `sweep` → `badge-type-sweep`, `routine` → `badge-type-routine`, else `badge-type-other`. A code comment in `DashboardPage.tsx` notes that these mirror the helpers in `components/Results/ResultsList.tsx` so the Dashboard's Type/Source/Status badges render with the **exact same palette** as the Results & Logs table.
- `statusBadgeClass(s)` — `completed` → `badge-success`, `failed` → `badge-error`, `cancelled` → `badge-cancelled`, else `badge-info`.

Query / repo rendering helpers:

- `parseQueryString(q)` — a regex-driven tokenizer (`/"([^"]*)"|'([^']*)'|(\S+)/g`) that splits a query string into parts, preserving double- and single-quoted spans.
- `formatKeywords(exec)` — prefers `exec.keywords` joined by `,` ; falls back to `parseQueryString(exec.query)` joined by `,` ; falls back to `–` .
- `formatRepos(exec)` — prefers `exec.repositories` joined by `,` ; falls back to `–` .

UI State Model

Local state (all in `DashboardPage`):

Name	Type	Role
`routines`	`Routine[]`	Full routine list from `/api/routines` .
`executions`	`Execution[]`	The 10 most recent executions from `/api/executions?limit=10` .
`exportPath`	`string`	Server-returned path for the most recent export; drives the success banner.
`exportError`	`string`	Error message from a failed export call; drives the error banner.

Derived values:

- `activeRoutines = routines.filter((r) => r.is_active)` — drives the Active Routines table.
- Per Recent Activity row: `isRunning` , `isCancelling` , `isActive` , `loc` , `isCloud` — derived from `activeExecution` and `e.execution_location` .
- `revealLabel` — `Reveal in Finder` on macOS (`window.resmonAPI?.platform === 'darwin'`), otherwise `Reveal in File Explorer` .

Context-supplied state consumed but not owned by the page: `activeExecution`, `cancelExecution`, `completionCounter`.

Key Interactions and Events

- **Initial load and refresh.** The `useEffect` keyed on `completionCounter` runs on mount and every time `ExecutionContext` bumps the counter (on any terminal-state transition per `ExecutionContext.tsx`). Each run issues two parallel GETs (`/api/routines`, `/api/executions?limit=10`) and silently ignores errors (`.catch(() => {})`). This is how the page auto-refreshes when a background execution finishes.
- **`handleViewReport(id)`** — calls `navigate(` /results?exec=${id} `)`.
- **`handleExportOne(id)`** — clears both banners, posts `/api/executions/export` with `{ ids: [id] }`, sets `exportPath` on success or `exportError` on failure; each banner is auto-cleared after 10 s by a `setTimeout`.
- **`handleReveal()`** — if `exportPath` is set and `window.resmonAPI?.revealPath` is available, calls `window.resmonAPI.revealPath(exportPath)`.
- **Per-row active-execution branching.** For each Recent Activity row, if `activeExecution?.executionId === e.id` and `activeExecution.status` is `running` or `cancelling`, the action cell renders `View Monitor + Cancel`; otherwise it renders `View Report + Export`. The Cancel button calls `cancelExecution(e.id)` and disables itself while the status is `cancelling`, showing a `fw-spinner` and `Stopping…`.
- **Source badge.** A `Source` column shows `Cloud` (`badge-info`) when `e.execution_location === 'cloud'`, otherwise `Local` (`badge-type-other`); missing values default to `local`.
- **Name column.** A `Name` column on the Recent Activity table renders the saved-configuration name when the execution row carries one. The cell prefers `e.saved_configuration_name` (joined in by `_enrich_execution_row` from the `saved_configurations` table via the `executions.saved_configuration_id` foreign key), falling back to the row's `routine_name` for routine-fired rows, and finally to `Execution #{e.id}` when neither is available. Manual executions launched from a loaded saved config (Deep Dive / Deep Sweep `ConfigLoader`) and "Save as new config" actions both populate `saved_configuration_id` so the name surfaces here.
- **Results column.** Renders `"<new> new / <total> total"`, preferring `e.new_results` / `e.total_results` (enriched by the backend in `_enrich_execution_row`) and falling back to `e.new_result_count` / `e.result_count`.
- **Date column.** Renders `e.start_time.slice(0, 16).replace('T', ' ')`, so an ISO timestamp like `2026-04-23T14:05:12Z` is shown as `2026-04-23 14:05`; missing values render `–`.

Error and Empty States

- **API errors on load:** both GETs use `.catch(() => {})`. The page does not surface a toast or inline error; tables simply remain empty. (This is the observed behaviour in `DashboardPage.tsx`; whether a more visible error surface is desirable is out of scope for this doc.)
- **Empty `activeRoutines`:** renders `No active routines. Create one from the Routines page.` in muted text.
- **Empty `executions`:** renders `No recent activity.` in muted text.
- **Export error:** renders `exportError` inside a `form-error` div above the table; auto-cleared after 10 s.
- **Export success:** renders `Export saved to: <path>` inside a `form-success` div, with a `Reveal…` button on the right when `window.resmonAPI?.revealPath` is available; auto-cleared after 10 s.
- **Unknown execution type / status:** fall through to the default branches of `typeBadgeClass` (`badge-type-other`) and `statusBadgeClass` (`badge-info`).

Backend

API Endpoints

The Dashboard touches four backend endpoints (all defined in `resmon_scripts/resmon.py`):

Endpoint	Used For
`GET /api/routines`	Active Routines table source.
`GET /api/executions?limit=10`	Recent Activity table source.
`POST /api/executions/export`	Per-row export bundle.
`POST /api/executions/{exec_id}/cancel`	Indirect, via `ExecutionContext.cancelExecution` when a visible row is the running execution.

The page does not itself call `/api/executions/merged`, `/api/search/*`, settings routes, scheduler diagnostics, or cloud auth endpoints. `CloudSyncCard` owns any cloud-sign-in calls it makes; those are covered in the Cloud and Settings info docs.

Request and Response Patterns

`GET /api/routines` (defined at `resmon.py:765`) — returns the full routine list. Each row is enriched by the handler with `last_execution` (copied from `last_executed_at`) and `last_status` (resolved from the latest `executions` row with matching `routine_id`). The Dashboard's `Routine` TypeScript interface consumes `id`, `name`, `schedule_cron`, `is_active`, and `last_executed_at`; other fields are ignored on this page.

`GET /api/executions` (defined at `resmon.py:1048`) — accepts `limit` (1–500, default 50), `offset` (≥ 0), and optional `type`. The Dashboard requests `limit=10` with no type filter. The handler runs `get_executions(conn, limit, offset, execution_type)` and passes each row through `_enrich_execution_row`, which normalises `query`, `keywords`, `repositories`, and aliases `total_results` / `new_results` onto the row. The Dashboard's `Execution` interface consumes these enriched fields.

`POST /api/executions/export` (defined at `resmon.py:1168`) — accepts `ExecutionExport { ids: list[int] }`. The handler:

1. Fetches each execution row via `get_execution_by_id`; 404s if none of the requested ids exist.
2. Reads the `export_directory` app setting via `get_setting(conn, "export_directory")`. If set, expands and creates that directory, validating with `mkdir(parents=True, exist_ok=True)` (400 on failure); the output filename is `resmon_executions_<YYYYMMDDTHHMMSS>.zip`. Otherwise it uses a `tempfile.NamedTemporaryFile(suffix=".zip")` path.
3. Calls `_build_execution_zip(rows, out_path)` which stages each execution into an `execution_<id>/` folder containing the report, logs, and a manifest JSON, then zips the staging root.
4. Returns `{ "path": str(out_path), "count": len(rows) }`.

The Dashboard only consumes the `path` field.

`POST /api/executions/{exec_id}/cancel` (defined at `resmon.py:1265`) — used only indirectly. The Dashboard calls `cancelExecution(e.id)` from `ExecutionContext`, which owns the HTTP call and the subsequent SSE state transitions; those details belong in the Monitor info doc.

Persistence Touchpoints

Read-side:

- `routines` table via `get_routines(conn)`, plus a per-routine lookup against `executions` for `last_status`.

- `executions` table via `get_executions(conn, limit=10, …)`, passing each row through `_enrich_execution_row` which also references the `query`, `keywords`, and `repositories` columns and the `result_count` / `new_result_count` aggregates.
- `app_settings` via `get_setting(conn, "export_directory")` inside the export handler.

Write-side from the Dashboard's own surface:

- None directly on Active Routines or Recent Activity display. The export handler writes a `.zip` to the filesystem but does not mutate the database.
- `POST /api/executions/export` does not alter the `executions` row; it only reads it and copies its report/log artifacts into a bundle.

Execution Side Effects

- **Filesystem:** `POST /api/executions/export` writes a zip either to the user-configured `export_directory` or to a temp file. The bundle contains one `execution_<id>/` directory per selected execution, each carrying the report Markdown, log file, and a manifest JSON assembled by `_build_execution_zip`. `_build_execution_zip` is intentionally factored out of the export handler so the same bundle can be attached to routine-completion emails (see Routines / Settings info docs).
- **Scheduler:** The Dashboard does not schedule or cancel any scheduler jobs directly. Cancelling a row flows through `ExecutionContext → /api/executions/{exec_id}/cancel`, which sets cooperative-cancel state in the `ProgressStore` / `SweepEngine` (full details belong in the Monitor info doc).
- **SSE / Progress events:** The Dashboard itself does not subscribe to `/api/executions/{exec_id}/progress/stream`. It relies on `ExecutionContext` to observe terminal-state transitions, bump `completionCounter`, and trigger the Dashboard's `useEffect` refetch.
- **`execution_location` population:** `_enrich_execution_row` does **not** set `execution_location`. The only endpoint that sets it explicitly is `/api/executions/merged` (resmon.py:1093), which stamps `"local"` on rows from `get_executions` and `"cloud"` on rows from `get_cloud_executions`. Because the Dashboard uses `/api/executions`, the field is absent on its rows and the `Local` badge is driven by the frontend's `?? 'local'` default.
- **Email / cloud sync:** None are triggered from the Dashboard. `CloudSyncCard` surfaces cloud-account state but does not itself mutate on render.